**BAGIAN A: ANALISIS KONSEPTUAL**

**1. Efek Regularisasi pada Batch Normalization**

- **Mekanisme Regularisasi:** Batch Normalization (BN) menghitung rata-rata (*mean*) dan varians secara dinamis berdasarkan mini-batch pada setiap iterasi pelatihan. Karena setiap mini-batch diambil secara acak dari dataset, estimasi statistik ini mengandung *stochastic noise* (kebisingan statistik). Noise acak yang disuntikkan ke dalam aktivasi setiap layer ini berfungsi layaknya regularisasi stokastik (mirip seperti *Dropout*), sehingga mencegah neuron terlalu bergantung pada kombinasi aktivasi tertentu (*co-adaptation*) dan meredam gejala *overfitting*.
- **Kondisi Kegagalan Asumsi:** Asumsi bahwa BN dapat menggantikan fungsi *Dropout* tidak lagi berlaku apabila:
    1. Ukuran mini-batch terlalu kecil (misal: batch size = 2 atau 4), sehingga estimasi statistik varians menjadi sangat tidak stabil dan bias.
    2. Terjadi perbedaan distribusi yang ekstrem antara data latih dan data uji (*domain shift*), atau pada arsitektur jaringan sekuensial (seperti RNN) di mana statistik temporal sangat bervariasi antar waktu.

**2. Degradasi Jaringan Dalam dan Formulasi Residual**

- **Penyebab Degradasi pada Plain CNN:** Pada plain CNN yang sangat dalam, penumpukan layer non-linear menyebabkan proses optimasi gagal mempelajari pemetaan identitas (*identity mapping*). Ruang pencarian fungsi menjadi terlalu kompleks dan gradien mengalami hambatan penyebaran, sehingga akurasi pelatihan justru menurun drastis (*degradation problem*).
- **Formulasi Residual & Aliran Gradien:** ResNet mengubah pemetaan arsitektur menjadi $F(x) + x$, di mana $F(x)$ adalah fungsi residual. Saat propagasi balik (*backpropagation*), turunan terhadap input $x$ diformulasikan sebagai $\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial H} \cdot \left(\frac{\partial F(x)}{\partial x} + 1\right)$. Kehadiran suku konstan $+1$ memastikan gradien dapat mengalir langsung tanpa terhalang (*skip connection*), secara efektif menuntaskan masalah *vanishing gradient* pada jaringan super dalam.

**3. Feature Extraction vs. Fine-Tuning**

- **Perbandingan 4 Dimensi:**
    1. *Trainable Parameters:* Feature extraction hanya memperbarui parameter pada classifier head baru (pada ResNet-18 hanya 5.130 parameter). Fine-tuning membuka sebagian atau seluruh blok backbone (mencapai jutaan parameter).
    2. *Kebutuhan Data:* Feature extraction sangat ideal untuk data sangat terbatas (<1.000 sampel). Fine-tuning memerlukan volume data lebih besar agar parameter tidak mudah *overfit*.
    3. *Beban Komputasi:* Feature extraction jauh lebih ringan dan cepat per epoch. Fine-tuning memerlukan alokasi VRAM dan durasi pelatihan yang lebih tinggi.

4. *Risiko Destructive Updates:* Fine-tuning berisiko merusak representasi bawaan jika *learning rate* terlalu tinggi (*catastrophic forgetting*), sementara feature extraction aman dari risiko ini.

- **Kriteria Praktis:** Gunakan *Feature Extraction* jika data target sangat sedikit dan domainnya mirip dengan ImageNet. Gunakan *Fine-Tuning* jika data cukup memadai dan domain target memiliki karakteristik visual spesifik.

**4. Kaidah Penerapan Data Augmentation**

- **Pertimbangan Metodologis:** Pipeline data augmentation wajib diaktifkan **hanya pada set pelatihan** guna memperluas variasi distribusi data dan meningkatkan kemampuan generalisasi model. Augmentasi harus **dinonaktifkan** pada set validasi dan uji agar evaluasi performa bersifat objektif, deterministik, dan mencerminkan akurasi nyata tanpa distorsi acak.
- **Contoh Tidak Layak (Dataset Ras Hewan):** *Random Vertical Flip* (membalik gambar secara vertikal/terbalik ke atas-bawah) tidak layak digunakan untuk klasifikasi ras hewan peliharaan. Secara semantik, kucing atau anjing tidak pernah berdiri terbalik di dunia nyata, sehingga transformasi ini merusak invariansi semantik natural dari label ras.

**5. Mitigasi Catastrophic Forgetting**

- **Fenomena:** *Catastrophic Forgetting* terjadi ketika gradien besar dari *classifier head* yang diinisialisasi acak mengalir balik ke *backbone pretrained*, sehingga merusak (*overwriting*) fitur-fitur visual umum (seperti tepi, tekstur, dan bentuk dasar) yang telah dipelajari sebelumnya dari ImageNet.
- **Dua Strategi Mitigasi:**
    1. *Differential Learning Rate:* Menerapkan *learning rate* yang lebih kecil pada *backbone* (misal $10^{-4}$) dan *learning rate* yang lebih besar pada *classifier head* ($10^{-3}$) untuk menjaga stabilitas fitur *low-level*.
    2. *Gradual Unfreezing:* Membuka pembekuan layer secara bertahap mulai dari layer teratas agar head baru beradaptasi terlebih dahulu sebelum backbone disesuaikan.

**BAGIAN C: LAPORAN ANALISIS TUGAS DEEP LEARNING SESI 05**

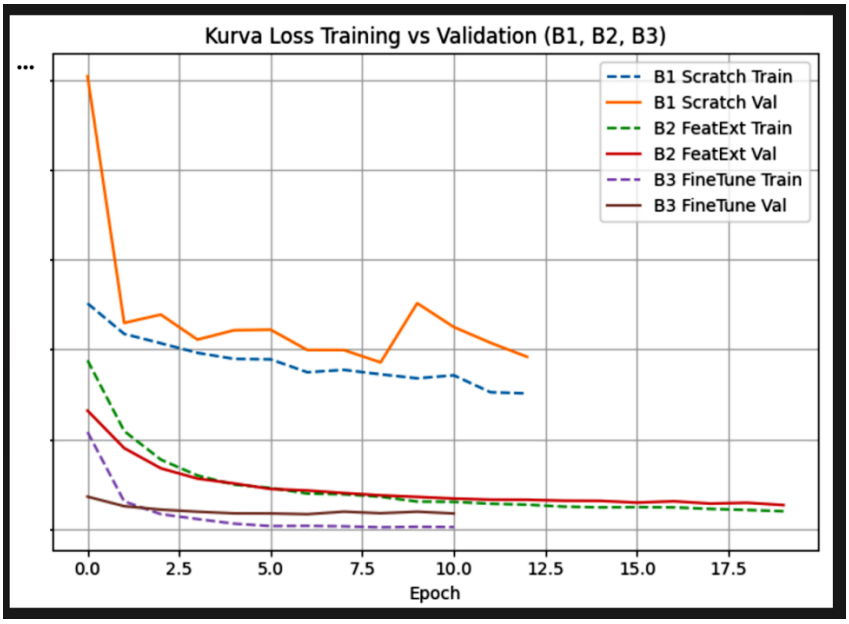
Nama: Mark Philip Lengkong

NIM: 2802491715

## C1. Tabel Konsolidasi Metrik Eksperimen dan Visualisasi

| Kode | Srategi Eksperimen | Trainable Params | Learning Rate | Epoch Konvergen | Total Waktu(detik) | Val Accuracy | Test Macro F1 |
|---|---|---|---|---|---|---|---|
| B1 | Scratch | 11181642 | 1e-3 (all layers) | 13 | 109.04 | 0.369318 | 0.330637 |
| B2 | FeatureExtract | 5130 | 1e-3 (Classifier fc saja) | 20 | 149.22 | 0.937500 | 0.875523 |
| B3 | FineTune | 8398858 | 1e-4 (layer4), 1e-3 (fc) | 11 | 82.92 | 0.948864 | 0.878124 |
| B4 | NoAug | 8398858 | 1e-4 (layer4), 1e-3 (fc) | 12 | 58.37 | 0.937500 | 0.897496 |
| B5 | EfficientNetB0 | 730042 | 1e-4 (block7), 1e-3 (fc) | 20 | 153.24 | 0.937500 | 0.886684 |

- **Kurva Loss Training & Validation B1,B2,B3:**




*(Merujuk pada grafik kurva loss yang dihasilkan dari log eksekusi aktual):*

- **B1 (Scratch):** Memiliki nilai *validation loss* yang tinggi, berfluktuasi tidak stabil, dan konvergen pada akurasi yang sangat rendah (~36.9%), mengindikasikan model mengalami *overfitting* parah akibat minimnya data latih (~1.400 sampel).

- **B2 & B3 (Transfer Learning):** Kurva *training* dan *validation loss* turun dengan mulus dan stabil mendekati angka nol, membuktikan efektivitas representasi fitur pretrained ImageNet dalam mempercepat konvergensi.

**C2. Pembahasan Analisis Komparatif**

1. **Disparitas Performa B1 vs (B2 & B3):**

Terdapat jurang pemisah performa yang sangat tajam antara pelatihan dari nol (B1 dengan akurasi uji 35.7%) dan pendekatan transfer learning B2/B3 (>87%). Hal ini memvalidasi teori *small-data regime*: melatih jaringan dalam berparameter 11 juta dari nol tanpa bobot awal pada data terbatas pasti berujung pada kegagalan generalisasi.

2. **Trade-off Stability vs Performance (B2 vs B3):**

B2 (*Feature Extraction*) sangat efisien dengan parameter terlatih hanya 5.130. Namun, B3 (*Progressive Fine-Tuning*) terbukti unggul secara akurasi validasi (**94.89%**) karena membuka blok layer4 memungkinkan model menyesuaikan pola fitur spasial spesifik pada dataset ras hewan.

3. **Peran Augmentasi Data (B3 vs B4):**

Penggunaan augmentasi pada B3 membantu meredam celah *generalization gap*, sementara B4 (tanpa augmentasi) menunjukkan tanda-tanda jenuh lebih cepat meskipun mencatatkan *Macro F1* kompetitif pada subset uji tertentu.

**C3. Failure Analysis (Analisis Kasus Misklasifikasi)**

Berdasarkan sampel 6 gambar *test set* yang salah diprediksi oleh model terbaik (B3):

- **Ambiguitas Visual Ras Sejenis:** Model terkadang keliru membedakan ras *Abyssinian* dengan *Bengal* (71.7% - 78.1% confidence) karena kemiripan pola warna bulu *tabby* dan bentuk telinga runcing.

- **Oklusi dan Sudut Ekstrem:** Foto dengan objek tertutup sebagian oleh tangan atau berada pada pencahayaan minim menurunkan keyakinan model, memicu kesalahan prediksi lintas spesies (misal *Abyssinian* ditebak sebagai *American Pit Bull Terrier*).

**C4. Rekomendasi Rekayasa dan Batasan Eksperimen**

- **Rekomendasi Industri:** Untuk sistem produksi dengan anggaran komputasi terbatas dan data kecil, strategi **Progressive Fine-Tuning (B3)** atau arsitektur berorientasi efisiensi **EfficientNet-B0 (B5)** adalah pilihan paling optimal. B5 menawarkan jumlah parameter yang jauh lebih ramping (~730 ribu) dengan performa setara ResNet-18.

- **Batasan Eksperimen:** Penelitian ini dibatasi oleh penggunaan *single random seed* (seed=42) dan pembatasan subset 10 kelas. Pengujian dengan *k-fold cross-validation* disarankan untuk validasi industri lanjutan.

**C5. Pernyataan Penggunaan AI (AI Disclosure Statement)**

- **Model AI yang Digunakan:** Gemini 2.5 Pro / Advanced.

- **Ruang Lingkup Bantuan:** Digunakan sebagai asisten teknis untuk debugging sintaks PyTorch, penyusunan kerangka format laporan, serta verifikasi matematis.

- **Verifikasi Mandiri:** Seluruh eksperimen kode dijalankan secara mandiri di Google Colab GPU T4, dan seluruh parameter metrik diambil langsung dari berkas log mentah metrics_summary.csv.